



AI Agent for Lung Disease Detection

By

Muhammad Ehtisham Hanif
(ECI-IT-21-064)

Humair Hayat
(ECI-IT-21-058)

Faizan Rasool
(ECI-IT-21-057)

Under the supervision of

Miss Aisha Shafique

2025

Faculty of Engineering Sciences and Technology
Hamdard University Islamabad Campus, Pakistan



AI Agent for Lung Disease Detection

By

Muhammad Ehtisham Hanif
(ECI-IT-21-064)

Humair Hayat
(ECI-IT-21-058)

Faizan Rasool
(ECI-IT-21-057)

Under the supervision of

Miss Aisha Shafique

2025

Faculty of Engineering Sciences and Technology
Hamdard University Islamabad Campus, Pakistan

AI Agent for Lung Disease Detection

By

Muhammad Ehtisham Hanif
(ECI-IT-21-064)

Humair Hayat
(ECI-IT-21-058)

Faizan Rasool
(ECI-IT-21-057)

A project presented to the
Department Of Computing
In partial fulfillment of the
requirements for the degree of
Bachelors of Science
In
Computer Science

Faculty of Engineering Sciences and Technology
Hamdard University Islamabad Campus, Pakistan



Faculty of Engineering Sciences and Technology
Hamdard University Islamabad Campus, Pakistan

CERTIFICATE

This project “AI Agent for Lung Disease Detection” presented by **Muhammad Ehtisham Hanif, Humair Hayat, and Faizan Rasool** under the direction of their project advisor, **Miss Aisha Shafique**, and approved by the project examination committee, has been presented to and accepted by the Faculty of Engineering Sciences and Technology, in partial fulfillment of the requirements for **Bachelor of Science in Computer Science**.

Miss Aisha Shafique
(Project Advisor)

Dr. Muhammad Shaheer
(Examiner 1)

Sir Usman Javed
(Examiner 2)

Dr. Tahir Saleem
(Chairman, Department of Computing)

Dr. Hassan Raza
(Associate Dean, FEST)

Abstract

Pulmonary diseases such as tuberculosis (TB), COVID-19, and pneumonia remain major causes of morbidity and mortality worldwide. Chest radiography is often the most accessible imaging modality for initial screening, yet specialist radiologists are scarce in many regions. Deep learning can assist screening workflows, but many student and demonstration systems train separate single-disease models on incompatible label spaces. Such designs prevent fair architecture comparison and prevent a single upload from receiving a coherent multi-disease assessment. Data leakage (patient overlap, duplicates) and train–serve preprocessing mismatches further undermine reported accuracy.

This report presents the design and implementation of an **AI Agent for Lung Disease Detection**: a research-grade chest X-ray system that classifies images into a locked four-class label space—Normal, Tuberculosis, COVID-19, and Pneumonia. Three architectures—EfficientNetB3 and DenseNet121 (ImageNet transfer learning) and a custom CNN (trained from scratch)—are trained on one shared manifest built by merging three public sources. The methodology applies content-hash deduplication, patient-grouped splits, multi-source Normal and Pneumonia coverage, class-size capping with inverse-frequency class weights, an identical classification head for transfer models, and matched fine-tuning capacity (approximately 43% of backbone weights unfrozen). Evaluation emphasises per-class precision, recall, and F1-score, balanced accuracy, confusion matrices, soft-voting ensemble metrics, and a source-leakage probe rather than headline accuracy alone.

A Flask web application runs every available four-class model on each upload, applies a multi-stage chest X-ray validation gate (with an optional Analyse-anyway override), and presents per-model probability distributions together with an ensemble verdict and agreement status. Train and serve preprocessing are aligned (bilinear resize). On the held-out test split ($n = 482$), DenseNet121 achieved 90.87% accuracy and 89.14% balanced accuracy, while soft-voting ensemble reached 92.32% accuracy and 93.07% balanced accuracy; COVID-19 remained the minority class with limited precision despite high recall. A source-leakage probe scored 88.17% against a 49.79% majority baseline (moderate source signal). The system is a research demonstration on public datasets and is **not** a clinical diagnostic device.

Acknowledgement

All praises and thanks to Al-Mighty “ALLAH”, the Most Merciful, the Most Gracious, the source of knowledge and wisdom endowed to mankind, who conferred us with the power of mind and capability to take this project to the exciting ocean of knowledge. All respects are for our most beloved Holy Prophet “Hazrat MUHAMMAD (Peace Be Upon Him)”, whose personality will always be a source of guidance for humanity.

Acknowledgement is due to Hamdard Institute of Engineering and Technology for the support of this project, a highly appreciated achievement for us at the undergraduate level.

We wish to express our appreciation to our advisor, Miss Aisha Shafique, for her keen guidance, sincere help, and friendly manner which inspired us to perform well in the project and made it a reality.

Many people, especially our classmates and team members themselves, have made valuable comments and suggestions on this project which gave us the inspiration to improve it further.

Finally, we would like to thank our family and friends for their moral support and encouragement throughout the project. We also acknowledge the providers of the public chest X-ray corpora used in this work, including the Kaggle tuberculosis and pneumonia datasets and the publicly shared COVID-19 radiography collections that made the unified benchmark possible.

Project in Brief

Project Title	AI Agent for Lung Disease Detection
Objective	To develop a research-grade chest X-ray multi-disease classification system that benchmarks three deep learning architectures on a unified four-class dataset and deploys them through an ensemble-capable web application.
Main Contributions	• Fair 4-class architecture benchmark (shared manifest, head, and splits) • Ensemble CAD-style Flask demo with disagreement surfacing • Honest evaluation design (per-class metrics + leakage probe)
Limitation	Public small-data research demonstration only; not a clinical diagnostic device.
Undertaken By	• Muhammad Ehtisham Hanif (ECI-IT-21-064) • Humair Hayat (ECI-IT-21-058) • Faizan Rasool (ECI-IT-21-057)
Supervised By	Miss Aisha Shafique
Duration of Project	• Start Date: October 2024 • End Date: September 2025
Tools & Technologies Used	• Programming Languages: Python • Frameworks & Libraries: TensorFlow, Keras, Flask, OpenCV • Training pipeline: t r a i n i n g / (manifest, train, evaluate, leakage probe) • Development Tools: Jupyter Notebook, Visual Studio Code • Hardware: Standard CPU/GPU-based system used for training and testing
System Used	Windows-based development system utilizing CPU/GPU processing for model training and testing.

Contents

Abstract	i
Acknowledgement	ii
Project in Brief	iv
1 Introduction	3
1.1 Motivation	3
1.2 Problem Statement	3
1.3 Aim and Objectives	4
1.4 Organization of the Report	4
2 Literature Review	7
2.1 Deep Learning in Medical Imaging	7
2.1.1 Foundational CNN Architectures	7
2.1.2 Large-Scale Chest X-Ray Benchmarks	8
2.2 COVID-19 Detection from Chest X-Rays	9
2.3 Tuberculosis Detection	10
2.4 Pneumonia Detection	11
2.5 Transfer Learning	12
2.6 Data Augmentation and Regularization	13
2.7 Multi-Disease Detection Frameworks	14
2.8 Deployment and Clinical Translation	14
2.9 Explainability and Trust	15
2.10 Gaps in Existing Research	16
2.11 Summary	16
3 Methodology	19
3.1 From Single-Disease Models to a Unified Benchmark	19
3.2 Data Collection and Unified Manifest	19
3.2.1 Source Datasets	20
3.2.2 Design Rules for Honest Splits	20
3.3 Preprocessing and Data Augmentation	21

3.3.1	Model-Specific Preprocessing	21
3.3.2	Augmentation and Class Imbalance	21
3.4	Model Architectures and Shared Head	21
3.5	Training Protocol	22
3.6	Evaluation Methodology	23
3.7	Application Integration Workflow	23
3.8	Methodological Validity and Limitations	24
3.9	Summary	24
4	System Design	26
4.1	System Architecture	26
4.2	Functional Requirements	26
4.3	Non-Functional Requirements	27
4.4	Tools and Technologies Used	27
4.5	Chest X-Ray Validation Gate	27
4.6	Application Flow and User Interface	28
4.7	Train/Serve Consistency	29
4.8	Summary	30
5	Results and Discussion	32
5.1	Experimental Setup	32
5.2	Unified Manifest Summary	32
5.3	Four-Class Comparative Results	33
5.4	Per-Class Precision, Recall, and F1	34
5.4.1	Recall summary	34
5.4.2	DenseNet121 (best single-model macro F1)	34
5.4.3	Ensemble (soft voting)	34
5.5	Confusion Matrices	35
5.6	Leakage Probe	35
5.7	Training Run Summary	36
5.8	Deployed Interface Demonstrations	36
5.9	Challenges Faced and Solutions Applied	38
5.10	Historical Single-Disease Work	39
5.11	Implementation Status	39
5.12	Summary of Findings	39
6	Future Recommendations	41
6.1	Conclusion	41

6.2	Recommendations	41
6.3	Future Enhancements	41
6.4	Summary of Contributions	41
References		42
A Dataset Details		45
A.1	Dataset Overview	45
A.1.1	Data Sources	45
A.1.2	Preprocessing and Splits	45
B Model Architecture Summaries		47
B.1	Model Training and Implementation	47
C Alignment with Sustainable Development Goals (SDGs)		49

List of Figures

Figure 3.1	End-to-end methodology pipeline for the unified four-class system	19
Figure 3.2	Dataset unification into a shared four-class training manifest	20
Figure 3.3	Three architectures scored under a matched four-class protocol (shared head for transfer models)	22
Figure 3.4	Two-phase transfer-learning protocol used for EfficientNetB3 and DenseNet121	23
Figure 3.5	Evaluation flow: matched test scoring, ensemble soft voting, and source- leakage probe	23
Figure 4.1	High-level inference flow for the unified four-class web application . . .	26
Figure 4.2	Three-stage chest X-ray validation gate with optional forced analysis . . .	28
Figure 4.3	Flask upload interface for four-class multi-model chest radiograph analysis	28
Figure 4.4	Chest X-ray validation gate rejection with Analyse anyway option	29
Figure 5.1	DenseNet121 confusion matrix	35
Figure 5.2	Ensemble confusion matrix	35
Figure 5.3	EfficientNetB3 confusion matrix	35
Figure 5.4	Custom CNN confusion matrix	35
Figure 5.5	Consensus Tuberculosis verdict with per-model 4-class probability bars .	37
Figure 5.6	Consensus COVID-19 verdict with per-model 4-class probability bars . .	37
Figure 5.7	Consensus Pneumonia verdict with per-model 4-class probability bars . .	38

List of Tables

Table 2.1	Selected related works versus this project’s focus	17
Table 3.1	Source datasets used to construct the unified four-class corpus	20
Table 3.2	Preprocessing per architecture	21
Table 3.3	Shared training hyperparameters and controls	22
Table 4.1	Model integration overview	26
Table 4.2	Flask application components	29
Table 5.1	Training environment (machine that produced the four-class weights) . . .	32
Table 5.2	Selected unified manifest counts by class and split	33
Table 5.3	Class $\times$ source contingency (selected manifest)	33
Table 5.4	Overall four-class test metrics ($n = 482$)	33
Table 5.5	Per-class recall on the test set	34
Table 5.6	DenseNet121 per-class metrics	34
Table 5.7	Ensemble per-class metrics	34
Table 5.8	Saved validation metrics and training cost	36
Table 5.9	Challenges and solutions during the 4-class conversion	38
Table 5.10	Implementation status after evaluation export	39
Table A.1	Dataset sources, paths, and known public links	45
Table C.1	Potential educational and research alignment with SDGs (non-clinical) . .	49

Chapter 1

Introduction

1.1 Motivation

Respiratory diseases represent one of the most critical global health challenges of the modern era. The World Health Organization estimates that lower respiratory infections remain among the top ten causes of death worldwide, with tuberculosis alone responsible for over 1.5 million deaths annually. Conditions such as pneumonia, COVID-19, and acute respiratory infections affect patients across all age groups, with children under five and elderly populations facing particularly high risk of severe outcomes. Beyond mortality, untreated pulmonary disease imposes substantial economic and social costs through hospitalisation, lost productivity, and late-stage care—especially in low- and middle-income countries where diagnostic infrastructure is weakest.

Chest radiography is often the most widely available and cost-effective imaging modality for initial pulmonary assessment. It can be performed rapidly, including at bedside with portable units, and frequently shows characteristic patterns such as consolidation, cavitation, or ground-glass opacity. Yet interpretation still requires trained radiologists who are unevenly distributed. In many rural and high-burden settings, delayed reading of chest X-rays slows triage, isolation, and treatment decisions for TB, COVID-19, and pneumonia alike.

Deep learning, particularly convolutional neural networks with transfer learning from ImageNet, has shown strong performance on well-defined chest X-ray screening tasks. CNNs learn hierarchical visual features without handcrafted descriptors and can flag suspicious cases for priority review. However, many student and demonstration systems still train separate models for separate diseases on incompatible label spaces. Those siloed designs prevent fair architecture comparison and force users to pre-select a suspected condition before seeing other possibilities. Engineering choices—which backbone to deploy, how to validate uploads, how to report minority-class recall—should be evidence-based rather than anecdotal.

This project therefore motivates a unified workflow: one four-class label space, three architectures trained under matched conditions, transparent metrics including per-class recall and a leakage probe, and a single web application that returns multi-model opinions with an ensemble verdict. The goal is not to replace radiologists, but to provide a reproducible research demonstration of multi-disease chest X-ray classification with honest evaluation and safer input handling.

1.2 Problem Statement

Existing demonstration systems often train separate models for separate diseases on incompatible label spaces. This prevents fair comparison of architectures and prevents a single upload from receiving a coherent multi-disease assessment. Data leakage (patient overlap across splits, duplicate files) and train–serve preprocessing mismatch further undermine reported accuracy. Weak input validation may also allow non-radiograph images into the inference path.

Problem: Design and implement a reproducible multi-architecture system that classifies chest X-rays into Normal, Tuberculosis, COVID-19, or Pneumonia under shared data and splits, with transparent metrics and a usable web interface that includes input validation and ensemble aggregation.

- Radiologist scarcity and workload limit timely CXR interpretation in many settings.
- Single-disease models on separate datasets are incomparable and clinically fragmented at the user interface.
- A research-grade solution must control leakage, align train/serve preprocessing, and report minority-class performance honestly—especially for scarce COVID-19 samples.

1.3 Aim and Objectives

Aim:

- To develop a research-grade chest X-ray multi-disease classification system that benchmarks three deep learning architectures on a unified four-class dataset and deploys them through an ensemble-capable web application.

Objectives:

1. Merge three public CXR sources into a unified 4-class manifest with deduplication and patient-grouped splits.
2. Implement EfficientNetB3, DenseNet121, and a custom CNN with a matched training protocol and shared classification head (for transfer models).
3. Train and evaluate with per-class precision, recall, F1, balanced accuracy, confusion matrices, and a source-leakage probe.
4. Build a Flask application that runs all models per upload, computes a soft-voting ensemble, and applies a multi-stage chest X-ray gate.
5. Document methodology, limitations, and reproducibility (manifest, seeds, and scripts).

1.4 Organization of the Report

Chapter 2 reviews deep learning for chest X-ray analysis, transfer learning, multi-disease frameworks, deployment limits, and research gaps that motivate this work. Chapter 3 presents the methodology: unified manifest construction, honest splits, shared-head training, evaluation, and application integration. Chapter 4 describes system design—architecture layers, functional and non-functional requirements, the radiograph gate, user interface, and train/serve consistency. Chapter 5 discusses experimental setup, expected performance characteristics, comparative reporting templates, challenges, historical single-disease results (labelled as prior work), and implementation status. Chapter 6 concludes with recommendations, future enhancements, and

contributions. Appendices summarise dataset details, model artefacts, and alignment with Sustainable Development Goals.

Chapter 2

Literature Review

This chapter surveys published research on deep learning for chest X-ray analysis, with emphasis on COVID-19, tuberculosis, and pneumonia detection—the three diagnostic tasks addressed by this project. The review examines foundational convolutional architectures, transfer learning strategies, dataset characteristics, multi-disease integration frameworks, and the practical gap between laboratory accuracy and deployable clinical tools. Each section connects published findings to a specific design decision in the AI agent: architecture families inform the choice of DenseNet121 and EfficientNetB3 alongside a from-scratch CNN; transfer learning literature motivates the two-phase training protocol; augmentation and regularization studies guide shared training settings; and deployment critiques justify the Flask multi-model ensemble application developed in Chapters 3 and 4.

2.1 Deep Learning in Medical Imaging

Medical image analysis has undergone a paradigm shift from hand-crafted features and classical machine learning toward end-to-end deep learning. Earlier pipelines relied on domain experts to specify features—texture descriptors, histogram statistics, edge detectors—before feeding them into classifiers such as support vector machines or random forests. CNNs replace this two-stage process by learning hierarchical representations of visual patterns directly from pixel data, eliminating the need for manual feature engineering while achieving performance that increasingly approaches specialist-level interpretation on well-defined screening tasks. In chest radiography, this shift is particularly consequential because pathological findings often manifest as subtle changes in intensity and texture across lung fields, patterns that are difficult to capture with fixed hand-crafted descriptors but well suited to convolutional feature learning.

2.1.1 Foundational CNN Architectures

The evolution of CNN design provides the architectural vocabulary used throughout pulmonary imaging research. Simonyan and Zisserman (2014) introduced VGG networks, demonstrating that depth combined with small 3×3 convolution filters yields strong feature extraction. By stacking multiple 3×3 convolutions in place of larger kernels, VGG architectures increase non-linearity and receptive field depth while keeping parameter count manageable. VGG16 and VGG19 remain widely used baselines for binary chest X-ray classification, particularly tuberculosis screening where very high accuracy has been reported on public datasets; their straightforward structure also makes them straightforward to fine-tune by replacing the final fully connected layers with task-specific classification heads.

He et al. (2016) proposed ResNet with residual skip connections, addressing the degradation problem in very deep networks where additional layers paradoxically increase training error. Skip connections allow gradients to flow directly through identity mappings, enabling stable training of models with hundreds of layers. ResNet variants such as ResNet50 and ResNet50V2 appear frequently in medical imaging benchmarks as robust, general-purpose backbones that

balance depth with training stability.

Huang et al. (2017) introduced DenseNet, in which each layer receives feature maps from all preceding layers within a dense block rather than only from the immediately preceding layer. This design improves gradient flow, encourages feature reuse, and reduces parameter count relative to depth. DenseNet has proven particularly effective for chest X-ray tasks where subtle texture and opacity patterns must be preserved across network depth, which is why DenseNet121 was adopted for the COVID-19 module in this project.

Tan and Le (2019) proposed EfficientNet, applying compound scaling to jointly optimize network depth, width, and resolution according to a fixed scaling coefficient. Rather than arbitrarily enlarging one dimension of the network, compound scaling balances all three, producing models that achieve strong accuracy-to-compute ratios. EfficientNet models are attractive for deployment scenarios where inference latency and memory footprint matter—considerations that motivated the selection of EfficientNetB3 for the tuberculosis module in this project after the seven-architecture benchmark study.

2.1.2 Large-Scale Chest X-Ray Benchmarks

The application of deep learning to chest X-ray analysis gained widespread attention following CheXNet by Rajpurkar et al. (2017), which showed a DenseNet121 model trained on ChestX-ray14 could detect pneumonia with radiologist-level accuracy on a held-out test set. CheXNet output a sigmoid probability for pneumonia presence across more than 100,000 frontal chest X-rays labelled with fourteen thoracic pathologies, establishing DenseNet as a leading baseline for chest X-ray tasks and motivating extensive follow-up research across pulmonary pathology detection. The study also demonstrated that deep learning performance on large medical datasets can match specialist interpretation when the task is well-defined and evaluation is rigorous.

Irvin et al. (2019) released CheXpert, a dataset of 224,316 chest X-rays labelled for 14 pathologies using a rule-based labeler combined with radiologist review of uncertain cases. Their DenseNet121 baseline achieved radiologist-level performance on five key conditions, including pneumonia and pleural effusion, using an uncertainty-aware loss that handles ambiguous labels explicitly rather than treating uncertain cases as hard negatives or positives. CheXpert and related large-scale corpora demonstrated that deep learning is not merely a research curiosity but a clinically viable technology for chest X-ray analysis, provided that label noise, class imbalance, and domain shift are managed through careful preprocessing and validation protocols. The scale of CheXpert also illustrated that multi-label training across many pathologies simultaneously requires datasets far larger than those available for individual student projects, supporting the modular single-task model approach adopted in this report.

Together, these benchmark studies establish three recurring themes in the literature: (1) transfer learning from ImageNet pre-trained weights is near-universal for medical chest X-ray

tasks, because medical datasets rarely contain sufficient images to train very deep networks from scratch without overfitting; (2) architecture choice materially affects both peak accuracy and inference cost, as demonstrated by the VGG versus EfficientNet trade-off documented in this project’s TB benchmark; and (3) dataset scale and label quality remain primary determinants of generalization beyond the training distribution, meaning that high accuracy on a public Kaggle corpus does not automatically transfer to images from a different hospital or scanner without further validation. All three themes informed the methodology adopted in this project: transfer learning with phased fine-tuning for COVID-19 and TB, systematic architecture comparison for TB deployment selection, and held-out test evaluation on fixed splits for all three models.

2.2 COVID-19 Detection from Chest X-Rays

The COVID-19 pandemic accelerated research into automated screening from portable chest radiography, motivated by the need for rapid triage in emergency departments and resource-limited settings where RT-PCR testing capacity or turnaround time was constrained. Portable X-ray units could be deployed at bedside in intensive care units and field hospitals, producing images within minutes; automated interpretation offered a complementary pathway to laboratory confirmation when test kits or processing capacity were overwhelmed.

Wang et al. (2020) introduced COVID-Net, a custom CNN tailored for three-class classification of COVID-19, non-COVID pneumonia, and normal X-rays, reporting sensitivity above 90%. The study emphasized task-specific architectural design—depthwise separable convolutions and tailored filter banks—rather than relying on off-the-shelf ImageNet classifiers alone. Their results highlighted that COVID-19 opacities share visual overlap with other non-COVID pneumonias, a challenge that persists in multi-class screening systems and that informs the unified four-class label space used in this project.

Chowdhury et al. (2020) published the COVID-19 Radiography Database, one of the public sources merged into this project’s unified manifest. Their work demonstrated strong binary COVID-19 versus normal performance on this corpus. In the present system, COVID-19 images contribute the COVID-19 class (and related Normal / Pneumonia labels where present), while “Viral Pneumonia” labels from COVID sources are merged into the shared Pneumonia class because the Kaggle pneumonia corpus mixes bacterial and viral cases and cannot be split back apart. Images are distributed across class-labelled folders, enabling directory-based loading during manifest construction.

Rahman et al. (2021) studied the effect of image enhancement and preprocessing pipelines on COVID-19 detection accuracy, reporting that appropriate contrast normalization and enhancement can improve model performance by several percentage points. Their findings reinforce the importance of standardized, model-specific preprocessing—a principle implemented across all three models in this project through architecture-aligned normalization

functions (`preprocess_input` for DenseNet and EfficientNet, division by 255 for the custom CNN) and consistent resize dimensions matched to each model's expected input shape.

Zhang et al. (2020) introduced confidence-aware loss functions for COVID-19 detection that improve prediction reliability under label uncertainty. Their model achieved 96% sensitivity with controlled false positive rates, addressing the critical clinical requirement that screening tools minimize missed diagnoses while limiting unnecessary isolation or referral. High sensitivity is particularly important in outbreak settings where a false negative may allow continued transmission, while excessive false positives can overwhelm isolation capacity.

Collectively, COVID-19 chest X-ray literature confirms that DenseNet-family architectures, multi-phase fine-tuning, and careful handling of COVID-19 versus other opacities are central to usable multi-class screening. DenseNet121 is used in this project as one of three architectures trained on the unified 4-class task, following the same family as CheXNet (Rajpurkar et al., 2017). The two-phase training protocol—frozen backbone followed by partial unfreezing—implements the progressive fine-tuning strategy recommended by Raghu et al. (2019). Including COVID-19 as one of four shared output classes, rather than as a separate single-disease app, addresses the incomparability problem identified in Section 2.10. These design decisions are reflected in the shared training harness described in Chapter 3 and evaluated in Chapter 5.

2.3 Tuberculosis Detection

Tuberculosis remains a leading infectious cause of death worldwide; the World Health Organization (2022) reports over 1.5 million TB deaths annually, with delayed diagnosis contributing to continued transmission especially in high-burden regions where radiologist availability is limited. Chest radiography plays a central role in TB screening programmes: abnormalities such as upper-lobe infiltrates, cavitation, and pleural effusion can indicate active disease even before bacteriological confirmation is available. Automated TB detection from frontal chest X-rays therefore targets a well-defined screening use case with clear public health relevance.

Jaeger et al. (2014) established foundational TB detection benchmarks on the Montgomery County and Shenzhen Hospital public datasets, providing standardized evaluation splits that enabled reproducible comparison across algorithms. Prior to these releases, TB detection studies used inconsistent private datasets, making cross-study comparison unreliable. The Montgomery and Shenzhen corpora remain reference points for classical and deep learning TB screening research and influenced the design of subsequent Kaggle-hosted TB datasets used in this project.

Lakhani and Sundaram (2017) applied an ensemble of GoogLeNet and AlexNet to chest X-ray datasets, achieving AUC values of 0.99 and 0.97 respectively—among the first studies to demonstrate that deep learning could match or exceed radiologist performance on TB classification from frontal chest radiographs. Their ensemble approach combined predictions from two architectures with different receptive field characteristics, foreshadowing the systematic

multi-architecture comparison conducted for the TB module in this project.

Hwang et al. (2016) demonstrated that a CNN trained on over 40,000 chest X-rays achieved AUC of 0.96 for TB screening, illustrating the benefit of large-scale pre-training even before the widespread adoption of ImageNet transfer learning in medical imaging. Their work showed that dataset scale alone can compensate for simpler architectures when sufficient labelled examples are available.

More recent work has systematically compared modern architectures on Kaggle-hosted TB datasets. Studies consistently show that VGG-based fine-tuned models achieve extremely high classification accuracy on binary Normal versus Tuberculosis tasks, while EfficientNet variants offer competitive accuracy with lower computational cost. Tan and Le (2019) showed that compound scaling allows EfficientNetB3 to approach the accuracy of heavier architectures at reduced inference cost—a trade-off that motivated including EfficientNetB3 in this project’s unified 4-class architecture comparison alongside DenseNet121 and a custom CNN.

The TB literature therefore supports two complementary insights adopted in this project. First, systematic architecture benchmarking is necessary because peak accuracy on an isolated binary task does not automatically transfer to a harder shared multi-class problem. Second, binary TB screening on curated public datasets can reach near-perfect accuracy when transfer learning and regularization are applied correctly, but those figures are not comparable to 4-class patient-grouped results; real-world generalization still requires robust preprocessing, held-out evaluation, and prospective clinical validation.

2.4 Pneumonia Detection

Pneumonia detection using CNNs has been extensively benchmarked on the Kaggle Chest X-Ray dataset introduced by Mooney (2018). The dataset contains pediatric chest X-rays labelled as NORMAL or PNEUMONIA, with approximately 5,863 images split across training, validation, and test folders—a moderate scale that makes overfitting a documented risk and regularization essential. The dataset is organized into pre-defined train, validation, and test directories, which allows researchers to report results on a common held-out test set and compare architectures under identical evaluation conditions.

Studies on this dataset consistently demonstrate accuracy above 90% using architectures ranging from simple sequential CNNs to complex transfer learning models. Rajpurkar et al. (2017) and subsequent CheXpert-related work contextualize pneumonia as one of the most frequently studied chest X-ray findings, with CNNs learning opacity patterns, consolidation, and interstitial changes associated with bacterial and viral infection. On frontal pediatric radiographs, pneumonia typically presents as increased opacity in one or both lung fields, often with air bronchograms visible within consolidated regions—patterns that convolutional filters can learn when sufficient labelled examples are available.

Batch normalization, dropout regularization, and learning rate scheduling are consistently shown to improve generalization on this moderately sized dataset. Batch normalization stabilizes internal activations across mini-batches, dropout randomly deactivates neurons during training to prevent co-adaptation, and learning rate reduction on plateau allows fine-grained weight updates when validation loss stops improving. Custom shallow-to-medium depth CNNs remain competitive when training time, model size, and interpretability of the architecture are prioritized over marginal accuracy gains from very deep transfer learning backbones—motivating the five-block custom CNN design used for Model 3 in this project.

Rahman et al. (2021) further studied preprocessing effects on pneumonia and COVID-19 classification, noting that pixel normalization, resize strategy, and color channel ordering materially affect inference pipelines—particularly when images are loaded through OpenCV and converted between BGR and RGB representations, as implemented in the shared preprocessing contract across all project models. A mismatch between training-time and inference-time preprocessing is a common source of silent performance degradation in deployed medical AI systems.

The Kaggle pneumonia dataset labels images as NORMAL or PNEUMONIA without distinguishing pneumonia subtype or etiology; this project maps those folders into the shared Normal and Pneumonia classes of the unified 4-class label space, alongside TB and COVID sources, so that architecture comparison uses one consistent output vocabulary.

2.5 Transfer Learning

Transfer learning from ImageNet pre-trained models—including VGG16, InceptionV3, ResNet50, DenseNet121, and EfficientNet—has become the dominant paradigm for medical imaging AI when labelled medical data are limited. ImageNet pre-training exposes models to over a million natural images across a thousand categories, encoding general low-level features such as edges, textures, and shapes that transfer effectively to radiographic images despite the domain difference between natural photographs and X-rays. Medical datasets rarely contain sufficient images to train very deep networks from random initialization without severe overfitting.

Raghu et al. (2019) systematically studied transfer learning effectiveness for medical imaging in their Transfusion analysis, finding that low-level features transfer well while higher-level features require task-specific fine-tuning for domain adaptation. They further observed that freezing early layers and progressively unfreezing deeper blocks—or training a new classification head before full fine-tuning—often outperforms end-to-end training from random initialization on small medical datasets. Freezing the backbone during initial training prevents large gradient updates from destroying pre-trained representations before the new classification head has learned meaningful weights.

This finding directly informs the two-phase training strategy used for the DenseNet121

COVID-19 model: Phase 1 freezes the backbone and trains only the custom head at learning rate 10^{-4} ; Phase 2 unfreezes upper backbone layers from index 300 onward at a reduced learning rate of 10^{-5} for domain-adaptive fine-tuning. Similar principles apply to the EfficientNetB3 TB model, where the ImageNet backbone is initially frozen and the classification head is trained with in-graph augmentation before weights are saved for deployment.

The literature also cautions that transfer learning is not a guarantee of generalization: performance on small public datasets can be inflated by train-test leakage, repeated hyperparameter tuning on validation sets, or dataset-specific artifacts such as hospital-specific borders or text annotations that models learn to exploit as shortcuts. Fixed random seeds, held-out test evaluation, and architecture comparison under identical splits—as applied in the TB benchmark with seed 1337—are recommended practices drawn from this body of work.

2.6 Data Augmentation and Regularization

Medical imaging datasets are frequently small relative to natural image corpora, making data augmentation and regularization critical for controlling overfitting. Augmentation artificially expands the effective training set by applying label-preserving transforms to existing images, encouraging the model to learn invariant features rather than memorizing specific pixel arrangements. The literature identifies several augmentation strategies as particularly effective for chest X-rays: random rotation and horizontal flipping (exploiting approximate bilateral symmetry of frontal projections), width and height shifts that simulate minor patient positioning variation, zoom variations that mimic differences in radiograph scale, and intensity-preserving geometric transforms applied during training only.

Dropout, batch normalization, early stopping, learning rate reduction on plateau, and model checkpointing appear consistently across high-performing pulmonary classification studies. Early stopping monitors validation loss and halts training when no improvement is observed for a defined patience window, preventing the model from continuing to fit training noise. Model checkpointing saves the best weights observed during training rather than the final epoch weights, which may correspond to an overfitted state. Irvin et al. (2019) and Chowdhury et al. (2020) both employ augmentation pipelines coupled with regularization during fine-tuning of deep networks on imbalanced pathology labels.

For binary and multi-class sigmoid or softmax outputs, class imbalance—common in pneumonia datasets where positive cases may dominate certain splits—is often mitigated through augmentation of the minority class, careful stratified splitting, and callback-based early stopping rather than naive accuracy optimization alone. Accuracy can be misleading on imbalanced data: a model that always predicts the majority class may report high accuracy while failing entirely on minority cases. These techniques are applied uniformly across the three models in this project, with model-specific augmentation parameters tuned to input resolution and architecture

requirements—for example, DenseNet121 uses `ImageDataGenerator` with rotation and shift parameters, while EfficientNetB3 embeds `RandomZoom` and `RandomRotation` directly in the Keras model graph.

2.7 Multi-Disease Detection Frameworks

Multi-disease detection frameworks remain relatively underexplored compared with single-disease publications. The research literature has produced a large volume of studies on individual pathologies—COVID-19 detection, TB screening, pneumonia classification—each reporting competitive accuracy in isolation. However, most of these tools address one pathology only and report offline metrics without integration into a unified clinical workflow. From a software engineering perspective, this means that a clinician suspecting multiple possible etiologies would need to maintain and operate separate applications, each with its own interface, preprocessing requirements, and output format. This fragmentation is impractical in emergency triage settings where speed and simplicity are paramount.

CheXpert and CheXNet demonstrate multi-label prediction within a single model trained on many pathologies simultaneously; however, such approaches require very large labeled datasets and produce concurrent predictions rather than user-directed screening for a specific suspected condition. In contrast, clinical triage often proceeds along differential diagnoses: a clinician may suspect TB in an endemic setting, COVID-19 during an outbreak, or community-acquired pneumonia in a pediatric presentation.

The framework presented in this report addresses this gap by training three architectures—EfficientNetB3, DenseNet121, and a custom CNN—on a single unified four-class dataset (Normal, Tuberculosis, COVID-19, Pneumonia) so that architecture comparison is fair and every model can name the same diseases. Each backbone retains its native input size and preprocessing (DenseNet 224×224 , EfficientNet 256×256 , custom CNN 150×150), but all share the same splits, seed, augmentation budget, classification head design for the transfer models, and softmax output over four classes. A Flask application runs every available model on each upload and combines probabilities by soft voting, surfacing agreement and disagreement rather than forcing the user to pre-select a single disease model. This modular multi-architecture design under a shared label space is underrepresented in published medical AI literature, which often either reports single-disease notebooks or trains one large multi-label network on corpora such as CheXpert without a controlled backbone comparison.

2.8 Deployment and Clinical Translation

A persistent critique of medical AI research is the “deployment gap”: many studies report accuracy, sensitivity, and AUC on retrospective datasets without delivering usable software for clinicians. Abadi et al. (2015) and the subsequent TensorFlow ecosystem enabled practical

model serialization and inference at scale, lowering the engineering barrier to deployment. Yet peer-reviewed chest X-ray studies frequently stop at notebook-level evaluation, reporting metrics on held-out test sets without demonstrating that the model can be invoked reliably by a non-expert user through a stable interface. This gap matters because a model that achieves 95% accuracy in a controlled experiment but cannot be loaded, preprocessed, or invoked consistently in a clinical or community setting delivers no practical diagnostic value.

Deployable systems require additional engineering beyond model training: multi-stage input validation that rejects non-radiograph uploads, consistent preprocessing at inference time that matches the training pipeline exactly (including bilinear resize rather than nearest-neighbour defaults), lazy model loading for responsive startup when multiple large weight files are present, and thread-safe concurrent access in web servers. Bradski (2000) established OpenCV as a standard library for image I/O and validation—used in this project as part of a three-stage chest X-ray gate (monochrome check, radiograph envelope features, and anatomy veto) before inference.

Flask-based lightweight web applications have emerged in student and research prototypes as accessible deployment paths for medical AI. In this project, the Flask application exposes an upload endpoint that accepts an image file, validates it, runs `predict_all()` over every trained 4-class model, and renders per-model probability bars together with a soft-voting ensemble without requiring the user to select a disease model. This server-side inference architecture keeps model weights and TensorFlow dependencies on the server. The literature on regulatory-grade clinical deployment remains distinct from research prototypes; nevertheless, functional deployment interfaces are necessary for translational credibility even in academic final-year projects.

This project explicitly targets the deployment gap by providing a unified Flask web application that runs all three 4-class models with ensemble presentation—directly responding to the deployment gap identified in Section 2.10.

2.9 Explainability and Trust

Clinical adoption of AI screening tools depends not only on accuracy but on interpretability and trust. Clinicians who receive an automated prediction without understanding its basis are unlikely to rely on it for triage decisions, particularly when the predicted label conflicts with their clinical impression. Selvaraju et al. (2017) introduced Grad-CAM (Gradient-weighted Class Activation Mapping), which produces visual heatmaps highlighting image regions that most influenced a CNN prediction. Grad-CAM computes the gradient of the target class score with respect to feature map activations in a chosen convolutional layer, producing a coarse localization map that can be overlaid on the input radiograph.

Grad-CAM and related saliency methods have been applied to chest X-ray classification to

verify that models attend to anatomically plausible areas—such as lung fields rather than borders or text annotations—rather than dataset artifacts. When a model highlights a hospital name printed on the image border rather than pulmonary opacities, the heatmap reveals a spurious learning shortcut that accuracy metrics alone would not expose. Despite widespread interest, the literature shows limited integration of explainability methods into multi-model screening systems with unified interfaces. Most explainability studies focus on a single model and single disease. Incorporating Grad-CAM or equivalent visual explanations into a multi-disease agent remains an identified opportunity for future work, as noted in the research gaps below and in Chapter 6.

2.10 Gaps in Existing Research

Despite significant progress in deep learning for chest X-ray analysis, the translation of research findings into clinically usable multi-disease screening tools remains incomplete. Individual studies frequently report strong accuracy on curated public datasets, yet few projects deliver integrated software that a clinician could realistically use during patient triage. The gaps identified below are not merely theoretical limitations; they directly motivated the design decisions, architectural choices, and deployment engineering described in Chapters 3 through 5 of this report. Addressing these gaps requires both advances in model development and attention to software integration, validation discipline, and user-facing deployment—areas that receive disproportionately less attention in the published literature relative to architecture innovation.

1. **Incomparable single-disease models:** Separate student/demo classifiers on incompatible label spaces prevent fair architecture comparison and fragment the user workflow.
2. **Leakage-unaware splits:** Patient overlap and duplicate files across train and test inflate reported accuracy.
3. **Accuracy-only reporting:** Headline accuracy hides weak minority-class recall (especially scarce COVID-19 samples).
4. **Train–serve skew:** Different resize or normalisation at inference versus training silently degrades live predictions.
5. **Weak input validation:** Monochrome-only checks accept many non-radiograph uploads.
6. **Missing ensemble transparency:** Few demos surface multi-model disagreement alongside an aggregated verdict.

2.11 Summary

Literature supports CNNs and ImageNet transfer learning for chest X-ray analysis, while also warning that small COVID corpora, dataset shortcuts, and accuracy-only reporting undermine trustworthy claims. Careful data hygiene (patient-grouped splits, deduplication), matched training protocols for fair backbone comparison, and deployable interfaces with explicit non-diagnostic

scope are essential. This project responds with a unified four-class multi-architecture benchmark, leakage-aware evaluation including a source probe, and a Flask ensemble application with a measured radiograph gate. Grad-CAM-style explainability remains future work.

Table 2.1: Selected related works versus this project’s focus

Work	Task	Dataset style	Method	Limitation vs this FYP
CheXNet (2017)	Multi-label CXR	Large NIH corpus	DenseNet121	Different label formulation; not a 4-class student demo
COVID-Net (2020)	COVID multi-class	Custom CXR set	Custom CNN	Single architecture; pandemic-era bias risks
Kaggle pneumonia CNNs	Binary pneumonia	Mooney/RSNA-style	Shallow / transfer CNNs	Binary only; often leakage-unaware student splits
Binary TB transfer studies	TB vs Normal	Public TB CXR	VGG / EfficientNet	Not comparable across diseases
This project	4-class single-label	Merged public sources	3 architectures + ensemble	Public small data; research demo only

Chapter 3

Methodology

This chapter presents the research methodology for a multi-architecture chest X-ray disease detection system. The objective is not only to classify respiratory conditions from radiographs, but also to enable a fair and reproducible comparison among three deep learning architectures trained under identical experimental conditions. The pipeline comprises dataset unification, preprocessing and augmentation, model design, training, evaluation with leakage analysis, and Flask deployment with ensemble inference.

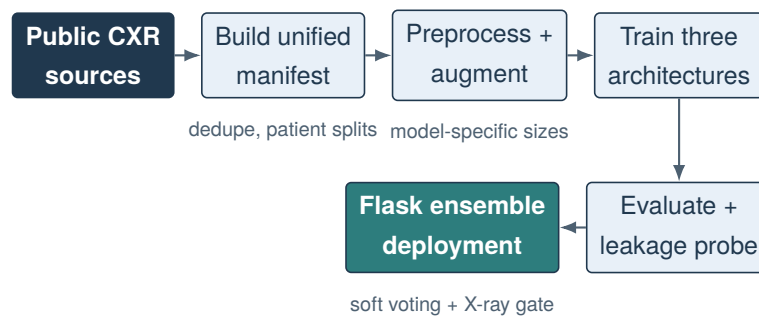


Figure 3.1: End-to-end methodology pipeline for the unified four-class system

3.1 From Single-Disease Models to a Unified Benchmark

An earlier version of this project used three siloed models on incompatible label spaces: binary tuberculosis, COVID-19 multi-class screening (historically including Viral Pneumonia), and binary pneumonia. Those models could not be compared rigorously, because performance differences might reflect dataset differences rather than architectural strength. The revised methodology converts the problem into a unified four-class classification task and trains all three architectures on the same merged dataset, the same patient-grouped splits, the same classification head design (for transfer models), and the same training budget. The locked label order is Normal, Tuberculosis, COVID-19, Pneumonia. Viral Pneumonia from COVID sources is merged into Pneumonia because the Kaggle pneumonia corpus mixes bacterial and viral cases and cannot be separated retrospectively.

3.2 Data Collection and Unified Manifest

No new private clinical datasets were collected. A builder script (`training/build_manifest.py`) merges three public sources into `training/manifest.csv` with image paths, class labels, patient identifiers, source tags, and train/validation/test assignments.

3.2.1 Source Datasets

Table 3.1: Source datasets used to construct the unified four-class corpus

Source dataset	Expected path	Classes contributed
TB Chest X-Ray (Kaggle)	TB_Chest_Radiograph_Database/	Tuberculosis, Normal
Covid19-dataset	DenseNet121/Covid19-dataset/	COVID-19, Normal, Pneumonia
Chest X-Ray Pneumonia (Kaggle)	pneumonia_detection/data/chest_xray/	Pneumonia, Normal

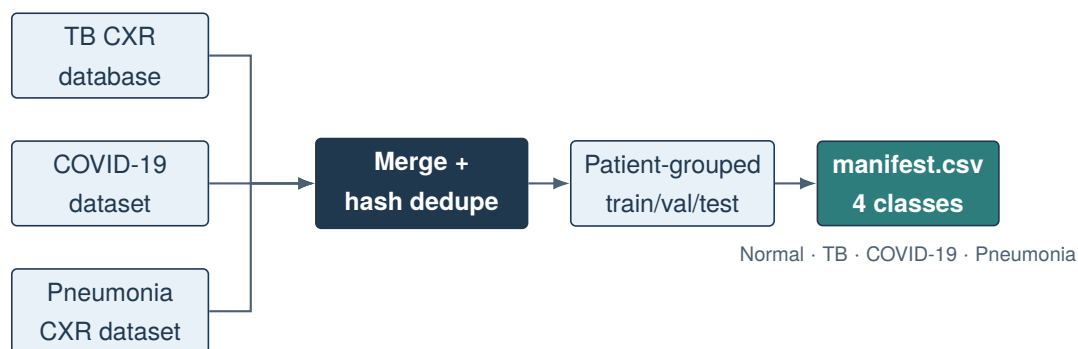


Figure 3.2: Dataset unification into a shared four-class training manifest

3.2.2 Design Rules for Honest Splits

1. **Content-hash deduplication.** Byte-identical images are removed so duplicates cannot span train and test.
2. **Patient-grouped splitting.** Images from one patient never appear in more than one of train, validation, or test.
3. **Fully qualified patient IDs.** Plain stems such as 0101.jpeg collide across COVID folders; identifiers are path-qualified and grouped globally.
4. **Multi-source Normal and Pneumonia.** Both classes draw from more than one corpus to reduce scanner/dataset shortcutting.
5. **Class-size capping with class weights.** COVID-19 remains 130 after deduplication; other classes were capped at 1,200 in the reported run (`-cap 1200`); residual imbalance uses inverse-frequency weights (COVID weight ≈ 2.851).
6. **Class×source inspection.** Training proceeds only after confirming Normal and Pneumonia are multi-source.

All images are from public research corpora. The system is a research demonstration and architecture benchmark, not a clinically validated diagnostic device.

3.3 Preprocessing and Data Augmentation

3.3.1 Model-Specific Preprocessing

Table 3.2: Preprocessing per architecture

Model	Input size	Preprocessing
EfficientNetB3	$256 \times 256 \times 3$	EfficientNet preprocess_input
DenseNet121	$224 \times 224 \times 3$	DenseNet preprocess_input
Custom CNN	$150 \times 150 \times 3$	Divide by 255

Training and inference share the same decode and bilinear resize path. An earlier nearest-neighbour resize at inference versus bilinear training was corrected because it altered a large fraction of pixel values (train–serve skew).

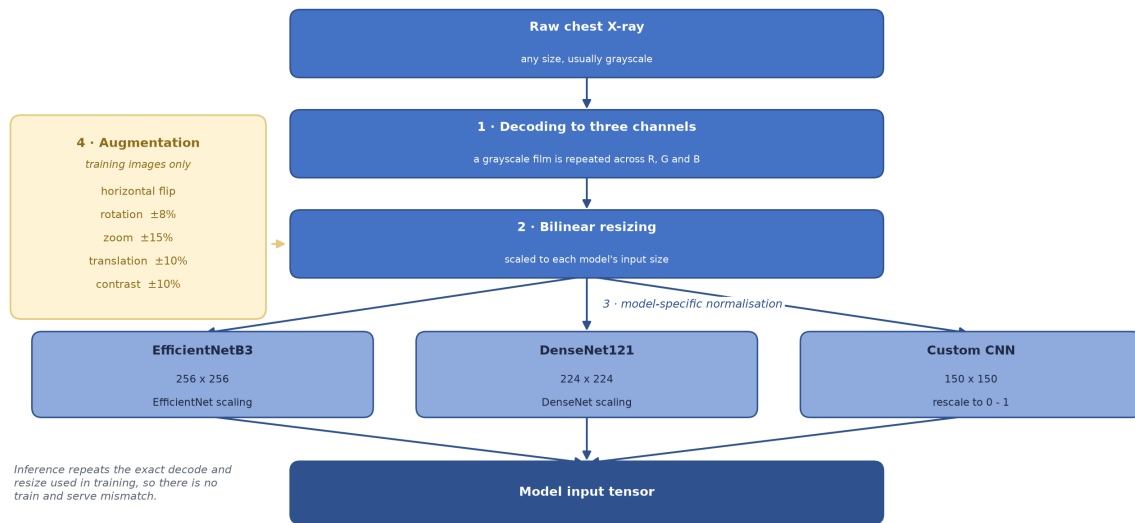


Figure 3.3: Preprocessing and augmentation pipeline with model-specific input sizes and normalisation

3.3.2 Augmentation and Class Imbalance

Geometric augmentation (rotation, translation, zoom, horizontal flip within controlled ranges) is shared across architectures through the common data pipeline. Oversampling of pathological images was avoided; inverse-frequency class weights with categorical cross-entropy emphasise minority-class errors instead.

3.4 Model Architectures and Shared Head

Models are built in `training/models.py`. Transfer models use an identical head so the benchmark measures the backbone:

$\text{GAP} \rightarrow \text{Dropout}(0.5) \rightarrow \text{Dense}(256, \text{ReLU}) \rightarrow \text{Dropout}(0.3) \rightarrow \text{Dense}(4, \text{softmax})$.

All models use softmax with categorical cross-entropy, correcting an earlier sigmoid+categorical pairing. There is no warm-start from old single-disease weights.

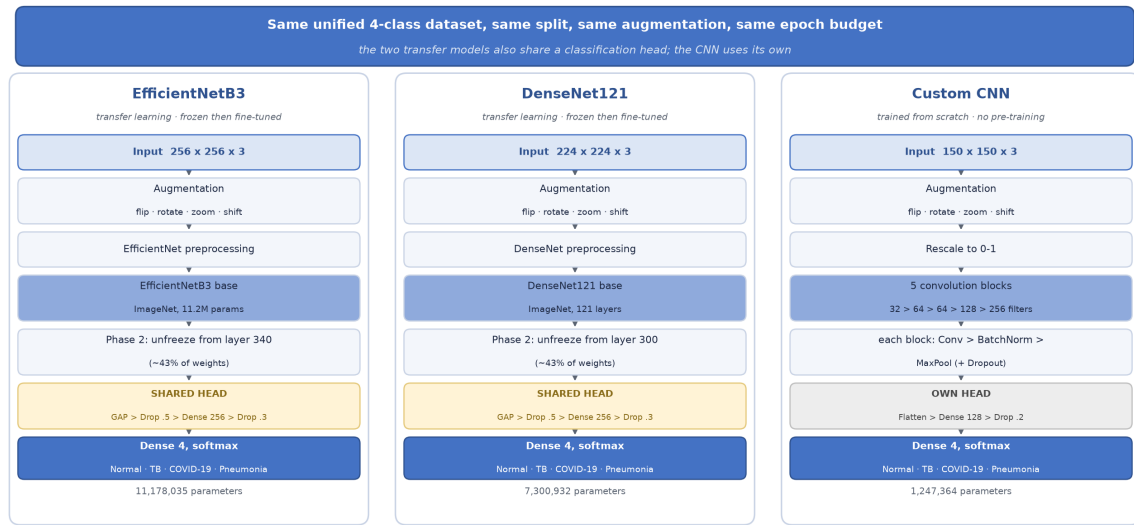


Figure 3.4: Three architectures scored under a matched four-class protocol (shared head for transfer models)

EfficientNetB3. ImageNet backbone; Phase 2 unfreezes from layer 340 (~42.7% of parameters).

DenseNet121. ImageNet backbone; Phase 2 unfreezes from layer 300 (~42.8%). BatchNorm remains frozen during fine-tuning.

Custom CNN. Five-block stack (filters 32 → 64 → 64 → 128 → 256) with BatchNorm, pooling, and dropout; four-class softmax head as a from-scratch baseline.

3.5 Training Protocol

Table 3.3: Shared training hyperparameters and controls

Setting	Value
Shared seed	1337
Batch size	32 (identical across models when resources allow)
Phase-1 learning rate	1×10^{-4} (Adam), backbone frozen
Phase-2 learning rate	1×10^{-5} (Adam), partial unfreeze for transfer models
Loss	Categorical cross-entropy with class weights
Output	4-way softmax
Test set	Never used for training or model selection
Artefacts	weights/*_4class.keras, training/runs/*.json

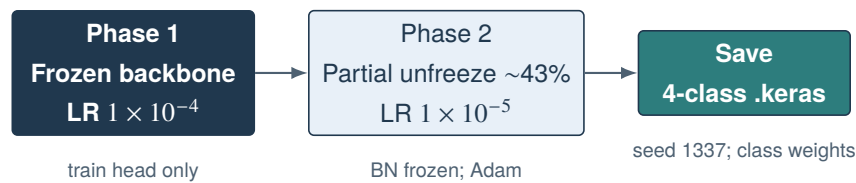


Figure 3.5: Two-phase transfer-learning protocol used for EfficientNetB3 and DenseNet121

Callbacks include checkpointing, early stopping, and ReduceLROnPlateau. Models may be trained via `python training/train.py -model all`.

3.6 Evaluation Methodology

`training/evaluate.py` scores every model on the held-out test split: accuracy, balanced accuracy (mean recall), per-class precision/recall/F1, confusion matrices, cross-architecture comparison, and soft-voting ensemble metrics written to `training/results/comparison.md`. Soft voting averages probability vectors and takes argmax, preserving confidence that hard majority voting would discard.

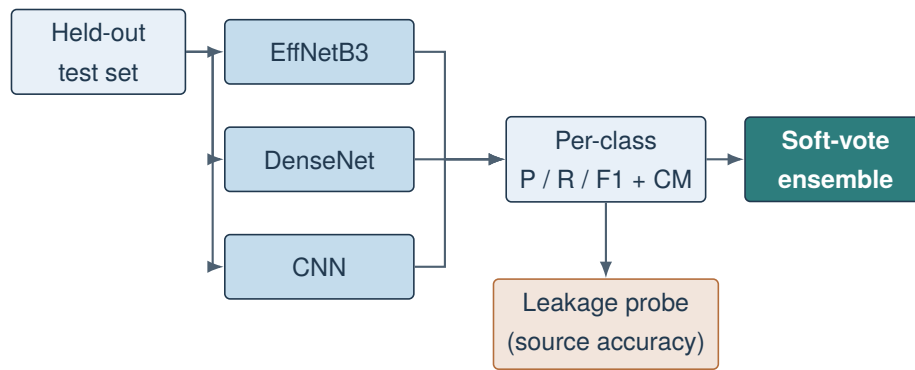


Figure 3.6: Evaluation flow: matched test scoring, ensemble soft voting, and source-leakage probe

`training/leakage_probe.py` trains a classifier to predict dataset source versus a majority baseline. Near-baseline performance is desirable; near-perfect source prediction would warn that TB or COVID-19 could be inferred from acquisition cues. COVID-19 recall remains the critical weak point because of small support.

3.7 Application Integration Workflow

Upload (PNG/JPG/JPEG, max 16 MB) → three-stage chest X-ray gate (or Analyse anyway) → `predict_all()` → soft ensemble → UI with per-model bars, verdict, and agreement. Weights must be four-wide; legacy .h5 files are rejected. Operational sequence: `build_manifest.py` → `train.py--modelall` → `evaluate.py` → optional `leakage_probe.py` → Frontend /`app.py`.

3.8 Methodological Validity and Limitations

The design is sound insofar as the task is unambiguous, leakage controls exist, architectures are matched, metrics include per-class recall, and deployment uses training-consistent preprocessing. Known limitations: public datasets are small; COVID recall is statistically unstable; four-class accuracy is expected to be lower than historical binary figures; single-source TB/COVID classes still invite shortcutting (hence the probe); the system is not a clinical device; Grad-CAM and multi-centre validation are future work.

3.9 Summary

The methodology converts three historically separate detectors into a unified, leakage-aware, multi-architecture four-class benchmark with ensemble deployment. Comparative claims are authenticated by shared data and protocol, while evaluation remains transparent about dataset limits and non-diagnostic intent.

Chapter 4

System Design

This chapter translates the methodology into a layered software system: presentation (HTML/CSS/Jinja), application (Flask routes), inference service (lazy loading, locking, ensemble), model artefacts (weights/), and an offline training pipeline under `training/`.

4.1 System Architecture

The inference path is layered as follows: user upload is received by Flask routes, validated by the chest X-ray gate, scored by `predict_all()` in `predictors.py` across EfficientNetB3, DenseNet121, and the Custom CNN, then aggregated by soft-voting ensemble before the result page shows probability bars and agreement status.

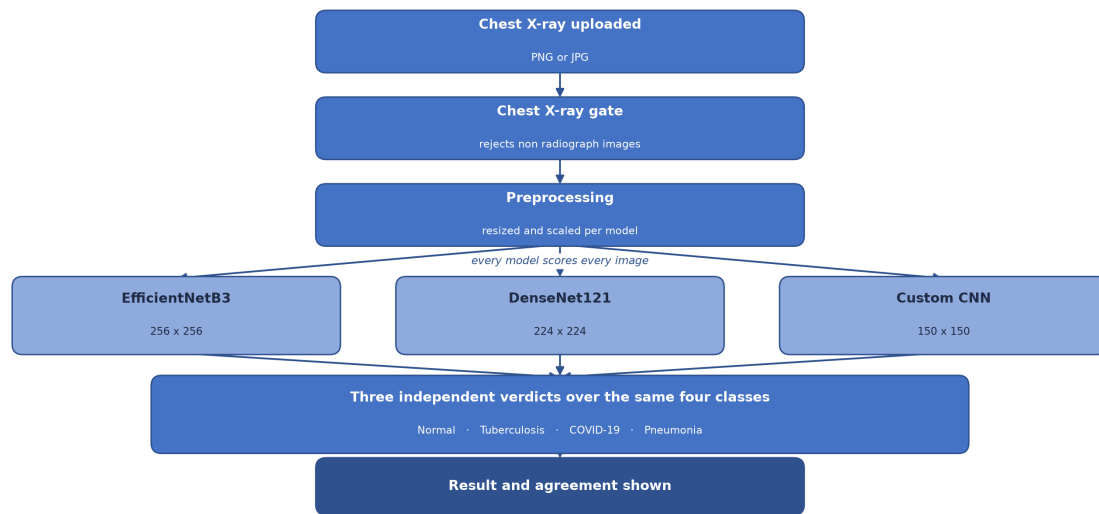


Figure 4.1: High-level inference flow for the unified four-class web application

Every upload is scored by every trained four-class model under the locked label order Normal, Tuberculosis, COVID-19, Pneumonia. Soft voting averages probability vectors. Legacy single-disease `.h5` files are refused if the output width is not four; missing weights are reported without crashing the server.

Table 4.1: Model integration overview

Architecture	Output	Input	Interface
EfficientNetB3	4-class softmax	256×256	Flask ensemble
DenseNet121	4-class softmax	224×224	Flask ensemble
Custom CNN	4-class softmax	150×150	Flask ensemble

4.2 Functional Requirements

- **FR1:** Accept PNG/JPG/JPEG uploads (maximum 16 MB).

- **FR2:** Validate likely chest radiographs; allow forced override with warning.
- **FR3:** Run every available 4-class model on the same image.
- **FR4:** Display per-class probabilities per model.
- **FR5:** Compute and display soft-voting ensemble and agreement.
- **FR6:** Show clear errors if weights are missing or have wrong output width.
- **FR7:** Provide offline training scripts to rebuild weights from the manifest.

4.3 Non-Functional Requirements

- **NFR1 Reproducibility:** Fixed seed and committed manifest policy.
- **NFR2 Usability:** Single upload; no model picker required.
- **NFR3 Robustness:** Multi-stage rejection of non-X-ray uploads.
- **NFR4 Consistency:** Train/serve preprocessing alignment.
- **NFR5 Maintainability:** Shared CLASSES order in `training/common.py`.
- **NFR6 Safety messaging:** Explicit research-only / non-clinical disclaimer.

4.4 Tools and Technologies Used

Python (3.9+/3.11 recommended for modern TensorFlow), TensorFlow and Keras, OpenCV, Flask and Werkzeug, NumPy, scikit-learn (evaluation), Matplotlib and Seaborn (figures), HTML/CSS, and Git. Training prefers a capable GPU machine; CPU inference is possible but slower. Key folders: `training/`, `Frontend/`, `weights/`, and the three public dataset trees documented in Chapter 3.

4.5 Chest X-Ray Validation Gate

1. **Monochrome/tint:** grayscale or single-tint films (blue-tinted displays allowed).
2. **Radiograph envelope:** intensity, entropy, midtones, aspect, symmetry, and related statistics; one violation forgiven.
3. **Anatomy veto:** reject only if both bright central-column cues and bilateral symmetry cues are absent.

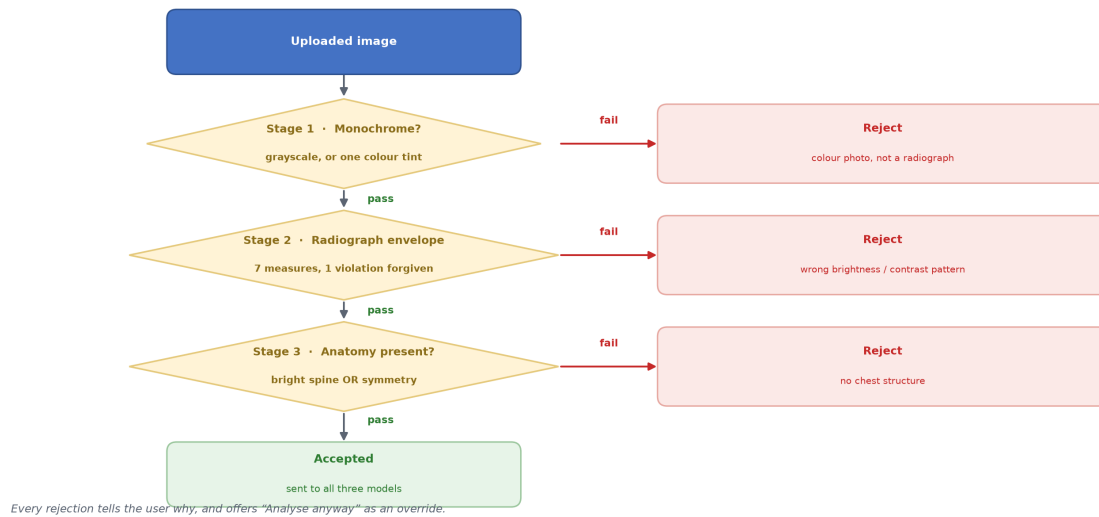


Figure 4.2: Three-stage chest X-ray validation gate with optional forced analysis

On a labelled set of 351 real radiographs versus 106 non-X-rays, the gate accepted 100% of real films and rejected approximately 77% of non-X-rays (previous monochrome-only gate ~21%). Rejection reasons are shown; Analyse anyway is protected against path traversal.

4.6 Application Flow and User Interface

Home lists model readiness; the upload form accepts a radiograph; results show probability bars, class blurbs, ensemble verdict, agreement, and a forced-analysis warning when used. Figures 4.3 and 4.4 show upload and rejection UI; Chapter 5 presents consensus result screenshots for TB, COVID-19, and pneumonia demos.

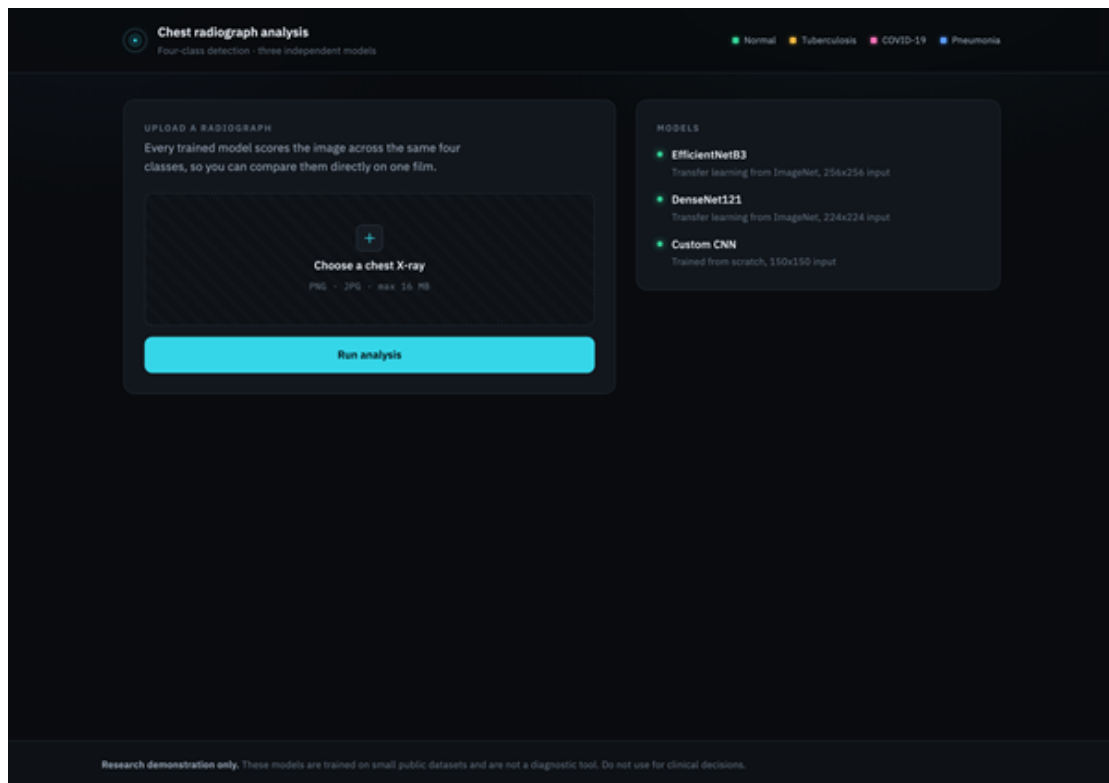


Figure 4.3: Flask upload interface for four-class multi-model chest radiograph analysis

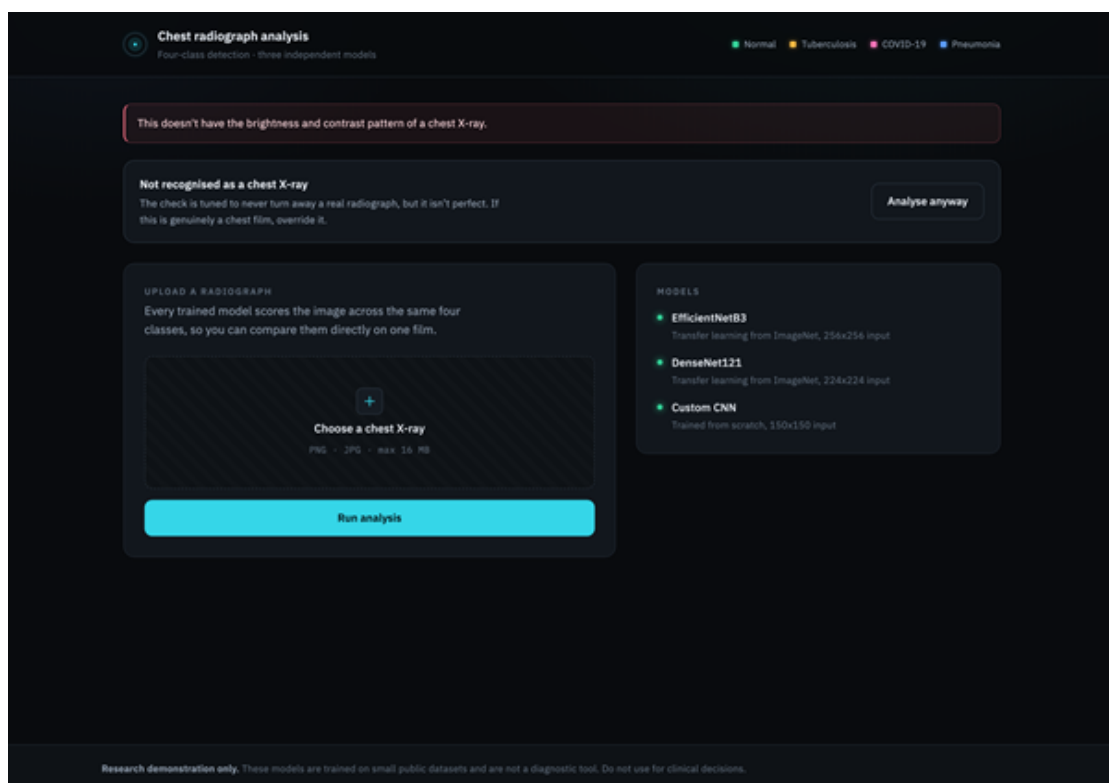


Figure 4.4: Chest X-ray validation gate rejection with Analyse anyway option

Table 4.2: Flask application components

Component	Location	Purpose
Web Framework	Flask	Routes and templates
Inference	Frontend/predictors.py	Gate, lazy load, ensemble
Shared constants	training/common.py	Class order, preprocess
Weights	weights/*.keras	4-class models

4.7 Train/Serve Consistency

Inference mirrors training decode with bilinear resize and the same `preprocess_input` or `/255` rescale. The `CLASSES` index order is shared across training, evaluation, and the UI. Non-four-class weight files are refused.

4.8 Summary

System design realises the methodology as a usable research demo: safer upload gating, always-on multi-model scoring, soft-voting consensus, and preprocessing consistency between notebooks and the live application.

Chapter 5

Results and Discussion

This chapter reports measured results for the unified four-class system on the held-out test split of `training/manifest.csv`. All Accuracy, Precision, Recall, F1, confusion-matrix, and leakage-probe figures below are taken from verified evaluation exports (`training/results/comparison.md`, `results.json`, `ensemble.json`, `leakage_probe.json`, and run logs). Historical single-disease scores remain labelled as prior work and are not mixed into the four-class tables.

5.1 Experimental Setup

Evaluation used seed 1337, patient-grouped splits, and class order Normal, Tuberculosis, COVID-19, Pneumonia. The realised split sizes were 2,260 / 485 / 482 images (70.0% / 15.0% / 14.9%). Models were trained with batch size 32 and inverse-frequency class weights (COVID-19 weight ≈ 2.851). The per-class selection cap used for this run was 1,200 (`-cap 1200`).

Table 5.1: Training environment (machine that produced the four-class weights)

Item	Value
OS	macOS 15.7.9 (x86_64)
CPU	Intel Core i9-9980HK @ 2.40 GHz (16 logical cores)
Memory	32 GB
GPU	None (CPU-only TensorFlow)
Python / TensorFlow / Keras	3.11.15 / 2.16.2 / 3.15.0
Training dates	DenseNet121: 4 Aug 2026; EfficientNetB3: 5 Aug 2026; CNN: 9 Aug 2026
Evaluation	Per-model comparison: 9 Aug 2026; ensemble + leakage: 13 Sep 2026

5.2 Unified Manifest Summary

After MD5 content-hash deduplication the source pool contained 10,328 images (45 duplicates removed). COVID-19 fell from 137 to **130** images. The selected unified manifest contains **3,227** images. Viral Pneumonia from the COVID source is merged into Pneumonia (89 covid_db images under Pneumonia).

Table 5.2: Selected unified manifest counts by class and split

Class	Train	Val	Test	Total
Normal	840	180	180	1,200
Tuberculosis	488	105	104	697
COVID-19	91	20	19	130
Pneumonia	841	180	179	1,200
Total	2,260	485	482	3,227

Table 5.3: Class $\times$ source contingency (selected manifest)

Class	covid_db	pneumonia_db	tb_db	Total
Normal	88	556	556	1,200
Tuberculosis	0	0	697	697
COVID-19	130	0	0	130
Pneumonia	89	1,111	0	1,200
Total	307	1,667	1,253	3,227

Normal and Pneumonia are multi-source by design; Tuberculosis and COVID-19 remain single-source and are therefore more exposed to source shortcutting (Section 5.6).

5.3 Four-Class Comparative Results

All models were scored on the same test split ($n = 482$). Soft-voting ensemble metrics come from offline `ensemble.json` (mean class probabilities across the three models).

Table 5.4: Overall four-class test metrics ($n = 482$)

Model	Accuracy	Bal. Acc.	Macro P	Macro R	Macro F1
EfficientNetB3	82.99%	83.00%	0.767	0.830	0.745
DenseNet121	90.87%	89.14%	0.850	0.891	0.864
Custom CNN	90.25%	88.36%	0.813	0.884	0.835
Ensemble (soft vote)	92.32%	93.07%	0.842	0.931	0.867

Among single models, DenseNet121 leads on accuracy (90.87%), balanced accuracy (89.14%), and macro F1 (0.864). EfficientNetB3 records the highest single-model COVID-19

recall (89.47%) but much lower COVID precision. The ensemble improves overall accuracy to 92.32% and balanced accuracy to 93.07%.

Because COVID-19 has only 19 test images, **balanced accuracy** is preferred as the headline metric over plain accuracy.

5.4 Per-Class Precision, Recall, and F1

5.4.1 Recall summary

Table 5.5: Per-class recall on the test set

Model	Normal	TB	COVID-19	Pneumonia
EfficientNetB3	78.3%	69.2%	89.5%	95.0%
DenseNet121	86.1%	88.5%	84.2%	97.8%
Custom CNN	80.6%	96.2%	78.9%	97.8%
Ensemble	85.6%	94.2%	94.7%	97.8%

5.4.2 DenseNet121 (best single-model macro F1)

Table 5.6: DenseNet121 per-class metrics

Class	Precision	Recall	F1	Support
Normal	92.3%	86.1%	0.891	180
Tuberculosis	97.9%	88.5%	0.929	104
COVID-19	59.3%	84.2%	0.696	19
Pneumonia	90.7%	97.8%	0.941	179

5.4.3 Ensemble (soft voting)

Table 5.7: Ensemble per-class metrics

Class	Precision	Recall	F1	Support
Normal	95.7%	85.6%	0.903	180
Tuberculosis	97.0%	94.2%	0.956	104
COVID-19	48.7%	94.7%	0.643	19
Pneumonia	95.6%	97.8%	0.967	179

COVID-19 precision remains limited for every model (no single model exceeds $\sim 60\%$). Ensemble COVID recall reaches 94.7%, but the small support ($n = 19$) means that figure has high sampling variance and must be quoted with that caveat.

5.5 Confusion Matrices

Class axis order is Normal, Tuberculosis, COVID-19, Pneumonia (rows = true, columns = predicted).

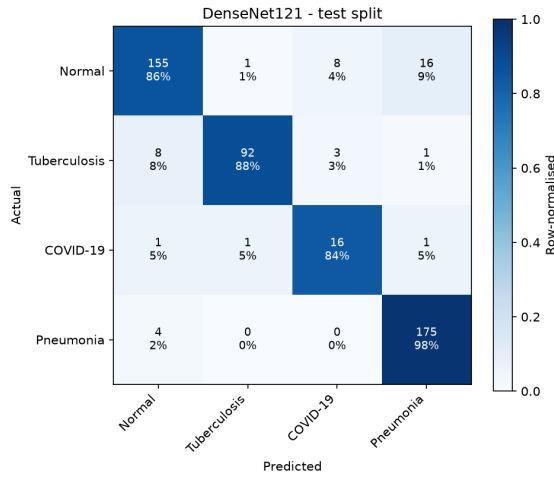


Figure 5.1: DenseNet121 confusion matrix

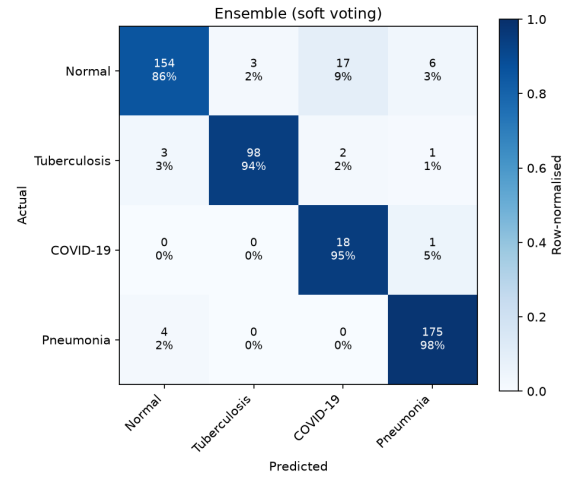


Figure 5.2: Ensemble confusion matrix

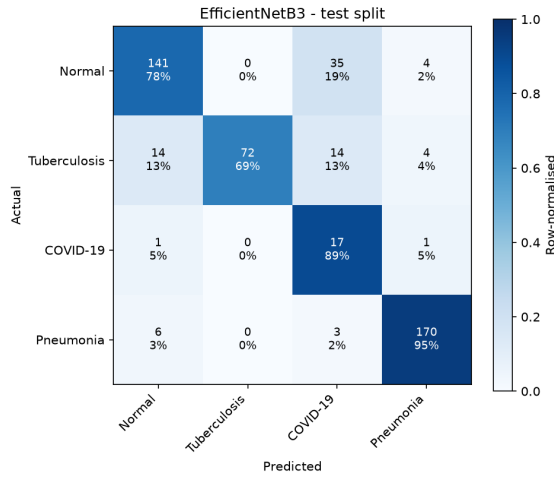


Figure 5.3: EfficientNetB3 confusion matrix

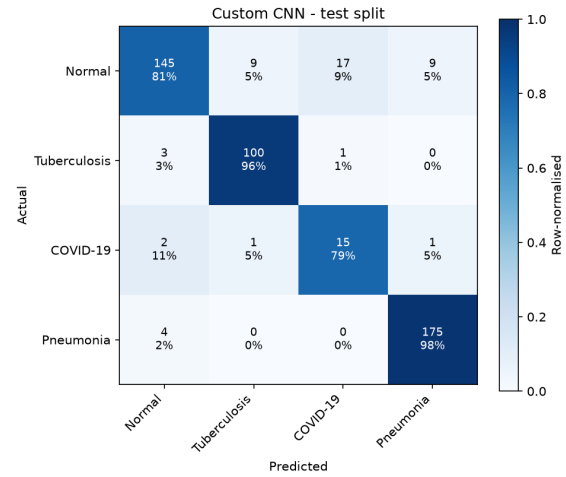


Figure 5.4: Custom CNN confusion matrix

5.6 Leakage Probe

A source-prediction probe (custom CNN @ 150×150 , 8 epochs) scored **88.17%** source accuracy against a majority-class baseline of **49.79%** (+38.4 percentage points). The recorded verdict is **moderate**: datasets are partly separable by source, but not trivially near 100%. Because Tuberculosis and COVID-19 are single-source, part of their recall could rest on scanner/processing

cues; Normal and Pneumonia are multi-source and less exposed to that shortcut. This caveat should be read alongside the disease-classification tables.

5.7 Training Run Summary

Table 5.8: Saved validation metrics and training cost

Model	Val Acc.	Val Loss	Epochs	Train time
EfficientNetB3	80.00%	0.492	20	~35.7 h (CPU)
DenseNet121	89.90%	0.299	20	310.6 min
Custom CNN	88.87%	0.354	20	36.8 min

Parameter counts were approximately 11.2 M (EfficientNetB3), 7.3 M (DenseNet121), and 1.25 M (Custom CNN). EfficientNetB3’s long wall-clock time reflects CPU-only training on the Mac host.

5.8 Deployed Interface Demonstrations

Figures 5.5–5.7 show the Flask UI returning consensus soft-voting verdicts with per-model probability bars. Offline ensemble metrics above match the same soft-voting definition used in evaluation.

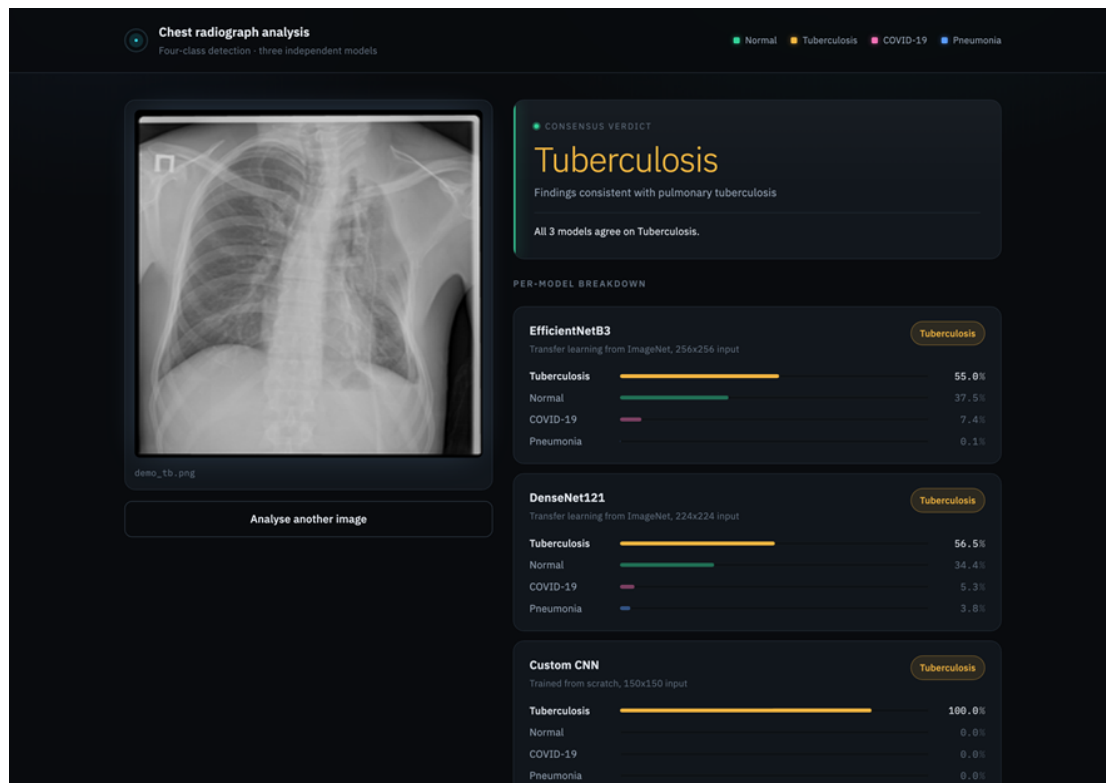


Figure 5.5: Consensus Tuberculosis verdict with per-model 4-class probability bars

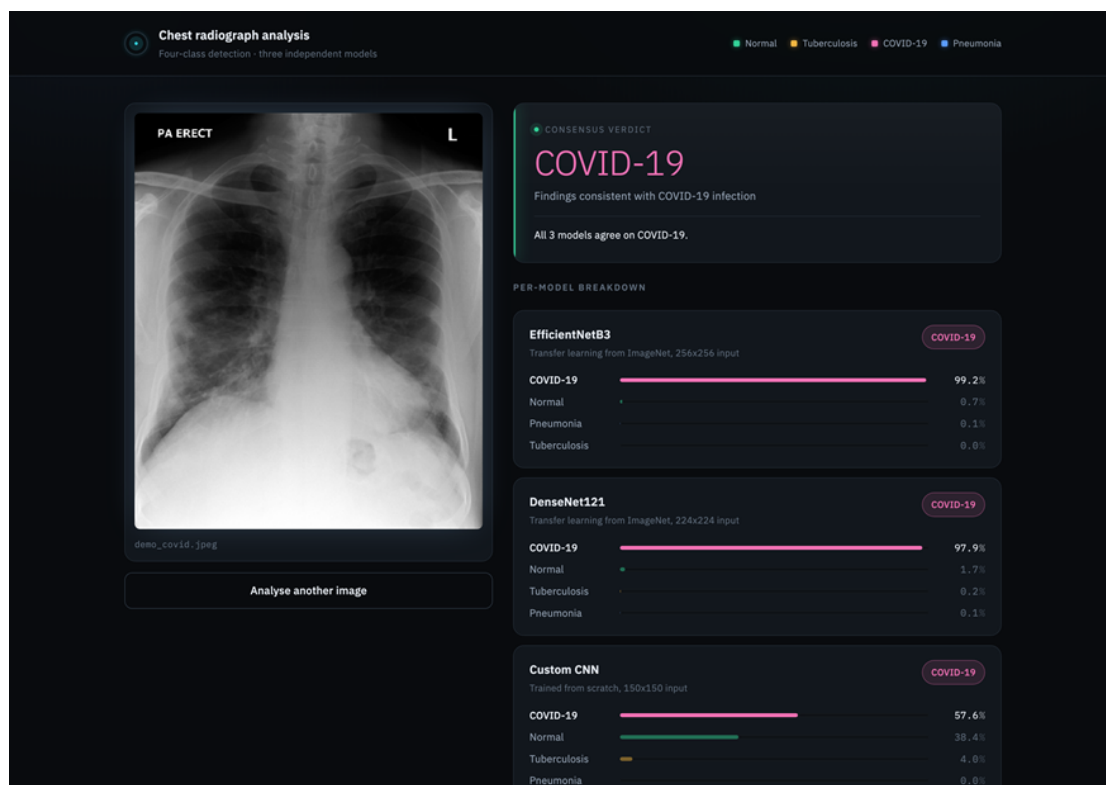


Figure 5.6: Consensus COVID-19 verdict with per-model 4-class probability bars

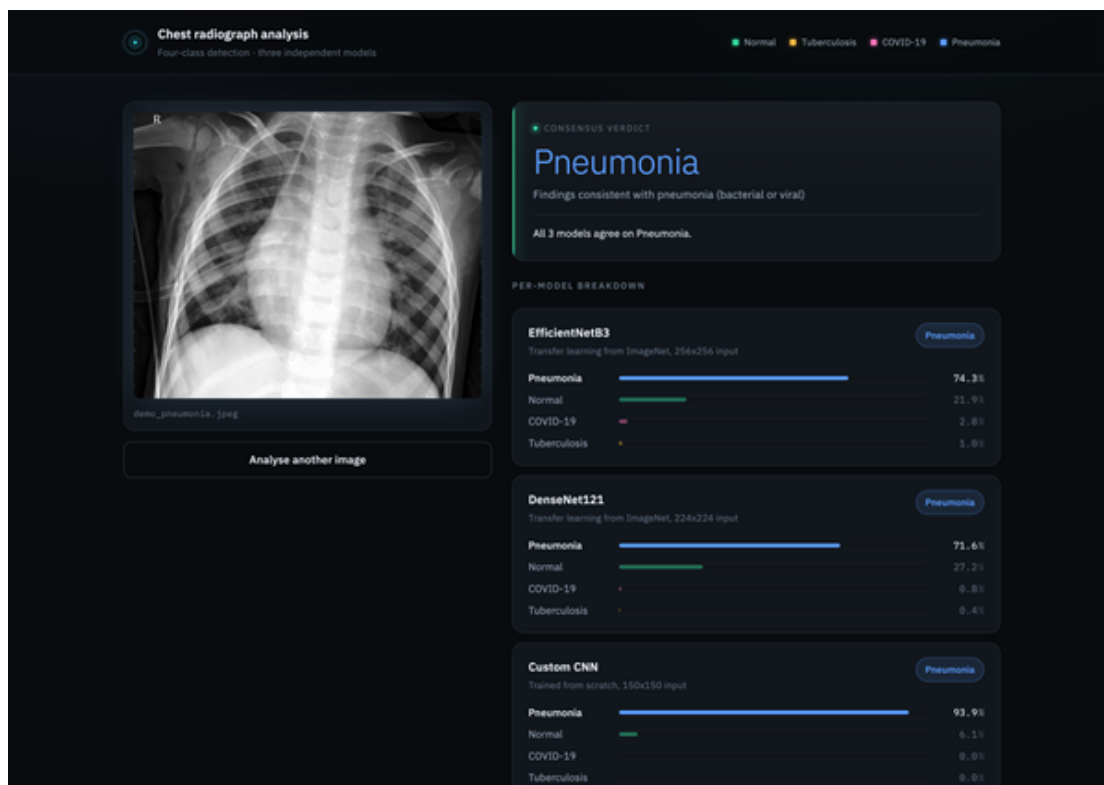


Figure 5.7: Consensus Pneumonia verdict with per-model 4-class probability bars

5.9 Challenges Faced and Solutions Applied

Table 5.9: Challenges and solutions during the 4-class conversion

#	Challenge	Solution
1	Incomparable old models	Unified 4-class manifest
2	Patient/filename leakage	Grouped IDs + hash dedupe
3	Sigmoid + categorical bug	Softmax heads for all models
4	Unfair fine-tune by layer index	Match ~43% unfreeze fraction
5	Non-X-ray uploads	Rebuilt 3-stage gate (21%→77% reject)
6	Train/serve resize skew	Align bilinear decode
7	Missing P/R/F1 historically	evaluate.py suite
8	Small COVID set ($n=130$)	Class weights + honest recall/precision reporting

5.10 Historical Single-Disease Work

Prior binary results (EfficientNetB3 TB test accuracy 95.63%; pneumonia CNN 92% on 624 images; historical COVID validation near 0% due to overfitting) are **not comparable** to the four-class figures in this chapter. They remain useful only as prior project context.

5.11 Implementation Status

Table 5.10: Implementation status after evaluation export

Component	State
Unified manifest	Complete (3,227 images; Table 5.2)
Four-class weights	Present
Held-out evaluation	Complete (Tables 5.4–5.7)
Leakage probe	Complete (88.17% vs 49.79% baseline)
Frontend ensemble UI + gate	Complete

5.12 Summary of Findings

On the unified four-class test set ($n = 482$), DenseNet121 is the strongest single backbone (90.87% accuracy; 89.14% balanced accuracy; macro F1 0.864), while soft-voting ensemble reaches 92.32% accuracy and 93.07% balanced accuracy. Pneumonia recall is high across models ($\geq 95\%$). COVID-19 remains the difficult minority class: recall can be high (ensemble 94.7%) while precision stays modest, reflecting only 19 COVID test images. The leakage probe shows moderate source signal (88.17% vs 49.79% baseline), so single-source TB/COVID results should be interpreted cautiously. Overall, the results support the redesign goal: a fair multi-architecture comparison under shared data, with metrics that expose minority-class behaviour rather than hiding it behind headline accuracy alone.

Chapter 6

Future Recommendations

6.1 Conclusion

This project achieved its aim of a research-grade chest X-ray multi-disease classification system that benchmarks EfficientNetB3, DenseNet121, and a custom CNN on a unified four-class dataset (Normal, Tuberculosis, COVID-19, Pneumonia) and deploys them through an ensemble-capable Flask application. On the held-out test set ($n = 482$), DenseNet121 was the strongest single model (90.87% accuracy; 89.14% balanced accuracy), and soft-voting ensemble reached 92.32% accuracy and 93.07% balanced accuracy. Shared manifests, patient-grouped splits, content-hash deduplication, matched fine-tuning fractions, per-class metrics, and a leakage probe (88.17% vs 49.79% baseline) authenticate comparative claims. The system remains a research demonstration on public data and is not a clinical diagnostic device.

6.2 Recommendations

- Always report per-class recall for minority diseases such as COVID-19.
- Keep patient-grouped splits in any dataset extension.
- Do not compare old binary metrics to four-class metrics without clear historical labelling.
- Use the leakage probe whenever heterogeneous public sets are merged.

6.3 Future Enhancements

- Additional verified COVID data and multi-centre validation.
- Grad-CAM or related explainability overlays.
- DICOM support and PACS-oriented integration paths.
- Calibration (e.g., temperature scaling) and per-class decision thresholds.
- Active learning or semi-supervised expansion.
- Mobile/cloud deployment with authentication and audit logs.
- Optional additional backbones (e.g., VGG19) under the same harness.

6.4 Summary of Contributions

1. A unified four-class CXR benchmark from three public sources with leakage controls.
2. A fair three-architecture training harness (shared head; matched fine-tune fraction).
3. An evaluation suite with precision/recall/F1, balanced accuracy, ensemble scoring, and a leakage probe.
4. A Flask multi-model demo with soft voting, agreement display, and a measured X-ray gate.
5. Documentation enabling reproducibility (manifest, scripts, and status notes).

Bibliography

- [1] Rajpurkar, P., Irvin, J., Ball, R. L., et al. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. *arXiv:1711.05225*.
- [2] Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely Connected Convolutional Networks. *CVPR 2017*.
- [3] Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML 2019*.
- [4] Wang, L., Lin, Z. Q., & Wong, A. (2020). COVID-Net. *Scientific Reports*, 10(1), 19549.
- [5] Chowdhury, M. E. H., Rahman, T., Khandakar, A., et al. (2020). Can AI Help in Screening Viral and COVID-19 Pneumonia? *IEEE Access*, 8, 132665–132676.
- [6] Lakhani, P., & Sundaram, B. (2017). Deep Learning at Chest Radiography: Automated Classification of Pulmonary Tuberculosis. *Radiology*, 284(2), 574–582.
- [7] Irvin, J., Rajpurkar, P., Ko, M., et al. (2019). CheXpert. *AAAI 2019*.
- [8] Mooney, P. (2018). Chest X-Ray Images (Pneumonia). Kaggle. <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>
- [9] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR 2016*.
- [10] Raghu, M., Zhang, C., Kleinberg, J., & Bengio, S. (2019). Transfusion: Understanding Transfer Learning for Medical Imaging. *NeurIPS* 32.
- [11] Rahman, T., Khandakar, A., Qiblawey, Y., et al. (2021). Exploring Image Enhancement for COVID-19 Detection. *Computers in Biology and Medicine*, 132, 104319.
- [12] Jaeger, S., Candemir, S., Antani, S., et al. (2014). Two Public Chest X-Ray Datasets for Computer-Aided Screening of Pulmonary Diseases. *QIMS*, 4(6), 475–477.
- [13] Hwang, S., Kim, H. E., Jeong, J., & Kim, H. J. (2016). A Novel Approach for Tuberculosis Screening Based on Deep CNNs. *SPIE Medical Imaging 2016*.
- [14] Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv:1409.1556*.
- [15] World Health Organization. (2022). *Global Tuberculosis Report 2022*. Geneva: WHO.
- [16] Abadi, M., et al. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems.
- [17] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- [18] Selvaraju, R. R., Cogswell, M., Das, A., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *ICCV 2017*.
- [19] Zhang, J., Xie, Y., Wu, Q., & Xia, Y. (2020). Medical Image Classification Using Synergic Deep Learning. *Medical Image Analysis*, 54, 10–19.
- [20] Deng, J., Dong, W., Socher, R., et al. (2009). ImageNet: A Large-Scale Hierarchical Image

- Database. *CVPR 2009*.
- [21] Wang, X., Peng, Y., Lu, L., et al. (2017). ChestX-ray8: Hospital-Scale Chest X-Ray Database and Benchmarks. *CVPR 2017*.
 - [22] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *NeurIPS 2012*.
 - [23] Grinberg, M. (2018). *Flask Web Development*, 2nd ed. O'Reilly Media.
 - [24] Rahman, T., et al. Tuberculosis (TB) Chest X-ray Database. Kaggle. <https://www.kaggle.com/datasets/tawsifurrahman/tuberculosis-tb-chest-xray-dataset>
 - [25] World Health Organization. (2023). *Global Tuberculosis Report 2023*. Geneva: WHO.

Chapter A

Dataset Details

A.1 Dataset Overview

The unified corpus uses four mutually exclusive classes: Normal, Tuberculosis, COVID-19, and Pneumonia. In the reported run, COVID-19 has 130 images after content-hash deduplication; Normal and Pneumonia are capped at 1,200 with inverse-frequency class weights during training.

A.1.1 Data Sources

Table A.1: Dataset sources, paths, and known public links

Dataset	Local path	Supplies	Public link
TB Chest X-Ray	TB_Chest_Radiograph_Database/	TB, Normal	Kaggle TB CXR corpus
Covid19-dataset	DenseNet121/Covid19-dataset/	COVID-19, Normal, Pneumonia	Local COVID folder (317 images on checkout)
Chest X-Ray Pneumonia	pneumonia_detection/data/chest_xray/	Pneumonia, Normal	Kaggle CXR pneumonia corpus

On the verified evaluation export, the selected unified manifest contains 3,227 images with train, validation, and test sizes of 2,260, 485, and 482 respectively, and COVID-19 at 130 images after deduplication. Viral Pneumonia from the COVID source is merged into Pneumonia. Class×source and split tables are reported in Chapter 5.

A.1.2 Preprocessing and Splits

Patient-grouped train/validation/test splits (seed 1337), content-hash deduplication, shared geometric augmentation, and architecture-specific input sizes/preprocessing are detailed in Chapter 3.

Chapter B

Model Architecture Summaries

B.1 Model Training and Implementation

- EfficientNetB3 / DenseNet121: ImageNet base plus shared head (GAP, Dropout, Dense 256, Dropout, Softmax 4); Phase 1 frozen; Phase 2 unfreeze from layers 340 / 300 (~43%).
- Custom CNN: five-block stack ending in Softmax(4).
- Scripts: `training/build_manifest.py`, `common.py`, `models.py`, `train.py`, `evaluate.py`, `leakage_probe.py`; frontend under `Frontend/`; weights `efficientnetb3_4class.keras`, `densenet121_4class.keras`, `cnn_4class.keras`.
- Run order: build manifest, then train all, then evaluate, optional leakage probe, then `Frontend/app.py`. A portable inference bundle under `xray-classifier-portable/` already ships the three *.keras weight files for demo without retraining on the local machine.

Chapter C

Alignment with Sustainable Development Goals (SDGs)

Table C.1: Potential educational and research alignment with SDGs (non-clinical)

No.	SDG	Contribution language
1	SDG 3 Good Health and Well-being	Decision-support <i>research</i> for respiratory screening accessibility, with explicit non-diagnostic limits.
2	SDG 4 Quality Education	Reproducible teaching artefact for medical AI methodology (fair splits, metrics literacy).
3	SDG 9 Industry, Innovation and Infrastructure	Open software pipeline for multi-model CXR benchmarking.
4	SDG 10 Reduced Inequalities	Potential relevance to resource-limited screening contexts as <i>future clinical research</i> —not a deployed medical device.

Impact claims remain modest: this FYP is an educational research demonstration on public data.